Université d'Ottawa
Faculté de génie

École de science
d'informatique
et de génie électrique

uOttawa
L'Université canadienne
Canada's university

University of Ottawa
Faculty of Engineering

School of Electrical
Engineering
and Computer Science

CSI2520 Programming Paradigms
CSI2520 Paradigmes de Programmation

The stable marriage problem and the Resident Matching Service
Comprehensive assignment
(32%)

Winter 2025

Individual project

Part 1 due February 2nd before 23:59

Part 2 due March 2nd before 23:59

Part 3 due April 6th before 23:59

Part 4 due le April 13th before 23:59

Late assignment policy: minus 10% for each day late. For example: a project due Friday night but handed out on the Monday morning: -30%

**** This project will count towards your final mark if and only if you obtain at least 50% on your exams (midterm + final)***

Le problème des mariages stables et le Service de Jumelage des Résidents
Projet intégrateur
(32%)

Hiver 2025

Ce projet est à réaliser de façon individuelle

Partie 1 due le 2 Février avant 23:59

Partie 2 due le 2 Mars avant 23:59

Partie 3 due le 6 Avril avant 23:59

Partie 4 due le 13 Avril avant 23:59

Pénalité de retard : perte de 10% de la note obtenue par jour de retard. Exemple : pour un projet dû le vendredi soir mais remis le lundi matin : -30%

**** Ce projet comptera dans votre note finale si et seulement si vous obtenez au moins 50% au cumul de vos examens (mi-session et final)***

Problem Description

In this assignment, we ask you to implement a solution to the **stable marriage problem**, an important problem that has applications in a variety of real-world situations. This solution will be based on the **Gale-Shapley algorithm**, named after its inventors, David Gale and Lloyd Shapley. Shapley and his colleague Alvin Roth applied this algorithm to help transplant patients to find donors; they obtain the 2012 Nobel prize in Economic Sciences for this achievement.

This is also the algorithm that is used by the Canadian Resident Matching Service (CARMS) to provide the mandatory matching service for medical residency training in Canada. Using this service, residents first submit the list of their preferred programs. The medical programs then receive the list of applicants and select them in order of preference. The objective of the stable matching algorithm is therefore to assign a program to every resident based on these preference lists. The challenge is to make sure that all the proposed matches are stable. In general, a match is considered to be stable if neither party to the match has a preferred party that also prefers them over their current match. In other words, in the case of resident matching, the solution should be such that there is no resident who would have preferred another program over the one assigned to them and for which that program would have also preferred this resident over some residents assigned to the program. One particularity of the CARMS context is that each resident must be matched with one program, but programs accept several residents (it's a many-to-one matching) as opposed to, for example, the organ donors matching problem which is one-to-one matching. Note also that when matching residents to programs, it may happen that some residents will not be matched, and some programs will not receive the number of residents requested.

The Gale-Shapley algorithm guarantees that the obtained solution will be a **stable matching**.

The algorithm

The Gale-Saley algorithm is simple to describe. Each party establishes their rank order list (ROL) that is the list of programs/residents in order of preference. The version used by CARMS is applicant-proposing. This means it tries to match the applicant to their first choice. If an applicant cannot be matched to their first choice program, the algorithm attempts to place the applicant into the second choice program of their ROL, and so on, until the applicant obtains a match, or all of the applicant's choices have been exhausted.

Each program has an associated quota (maximum number of residents it can accept). Programs accept applicants on its ROL, until the quota is reached. When new residents are proposed, it removes less preferred ones in favor of more preferred applicants. At the end of the process, each applicant should have been matched to their most preferred choice possible from their ROL. You can also have unmatched residents and programs that have not been able to reach their quota.

The stable matching can be found with the Gale-Shapley iterative algorithm. The corresponding pseudocode is given below. The input are two ROLs, $L_{\{resident\ preferences\}}$ and $L_{\{program\ preferences\}}$.

The algorithm produces a set of stable matches and a set of unmatched residents. In this pseudocode, an available resident is one that is not part of either of these two sets.

```
GaleShapley ( Residents, ROL_residents, ROL_programs )

Matches:= {}
Unmatched:= {}

while ( all residents have been matched or can't be matched )

          pick any available (not currently matched) resident r

               for all program p in ROL_residents(r)
                    if (r ∉ ROL_programs(p) ) then
                         continue to next program
                    else if (p has not reached its quota) then
                         Matches+= (r,p)
                         break for loop
                    else if (p prefers r
                      over the least preferred matched resident r')
                         Matches-= (r',p) // r' now becomes available
                         Matches+= (r,p)
                         break for loop

               if (r exhausted their ROL_residents(r)) then
                    Unmatched+= r

return Matches , Unmatched
```

A recursive solution

While Gale-Shapley is an iterative solution, the recursive McVitie-Wilson variant of the algorithm will be easier to implement in some paradigms. One can think of McVitie-Wilson as an alternating recursion of two functions: a function `offer` and a function `evaluate`. The recursive functions need to be called from a main loop which calls `offer` for each resident once.

McVitieWilson (Residents, $ROL_{residents}$, $ROL_{programs}$)

```
for all resident r
     r.MatchedProgram:= None
for all program p
     p.MatchedResidents:= {}

for all resident r
     offer( r )

offer ( resident r)

     find next available program p in ROL_residents(r)

     if (r exhausted their ROL_residents(r)) then
          r.MatchedProgram= None
     else
          evaluate( r, p )


evaluate ( resident r, program p)
     if (r ∉ ROL_programs(p) ) then
          offer( r )
     else if (p has not reached its quota) then
          r.MatchedProgram= p
          p.MatchedResidents+= r
     else if (p prefers r
               over the least preferred matched resident r')
          p.MatchedResidents-= r'
          p.MatchedResidents+= r
          r.MatchedProgram= p
          offer( r')
     else
          offer( r )
```

Input and Output files

To run your stable matching solution, you need the ROLs from both the residents and the programs. These are specified in CSV (comma-separated values) files. Each resident has a name and a unique numerical identifier. Programs have names and a three-letter unique identifier. Here is an example of csv file for the residents' ROLs.

```
id,firstname,lastname,rol
574,Salvatore,Williams,"[NRS,HEP,MMI]"
517,Rosalie,Frederick,"[NRS,MMI]"
403,Aspyn,Olson,"[HEP,NRS]"
226,Zev,Jarvis,"[MMI,HEP,NRS]"
126,Indie,Medrano,"[MMI,NRS]"
828,Emma,Tremmo,"[OBG,NRS]"
913,Camille,Paquet,"[OBG,MMI,HEP]"
773,Marie,Clown,"[OBG]"
616,Laurent,Robert,"[OBG,MMI,HEP,NRS]"
377,Tom,Tan,"[MMI,OBG]"
```

The last value in each line is the ROL listing program IDs in order of preferences. An example of csv file for the programs' ROLs is shown here.

```
id,name,numberOfPos,rol
MMI,Microbiology,1,"[574,517,226,913,377,126]"
HEP,Hematological Pathology,2,"[403,574,913,616,226]"
NRS,Neurosurgery,4,"[574,517,403,828,226,126]"
OBG,Obstetrics and Gynecology,3,"[616,828,773,913]"
```

The third value integer corresponds to the number of positions available (program's quota). The last value is the ROL listing the resident IDs in order of preference. A resident applying to a given program might not have been selected by the program, so it is possible that their ID does not appear in this list even if they applied to this program.

We give you three versions of these input files

- `residentSmall.csv` and `programSmall.csv`
  - these ones are just toy files that you can use to debug your programs
- `residents4000.csv` and `programs400.csv`
  - this is an example with 4000 residents and 35 programs for which we ask you to find a solution
- `residentsLarge.csv` and `programsLarge.csv`
  - this is an example with 50000 residents to be used only to test the concurrent version (Go) of your assignment

The output of your programs will be the list of matches using the format shown here. Here is the output describing a stable match set for the small input files.

```
lastname,firstname,residentID,programID,name
Clown,Marie,773,OBG,Obstetrics and Gynecology
Frederick,Rosalie,517,NRS,Neurosurgery
Jarvis,Zev,226,MMI,Microbiology
Medrano,Indie,126,NRS,Neurosurgery
Olson,Aspyn,403,HEP,Hematological Pathology
Paquet,Camille,913,HEP,Hematological Pathology
Robert,Laurent,616,OBG,Obstetrics and Gynecology
Tan,Tom,377,XXX,NOT_MATCHED
Tremmo,Emma,828,OBG,Obstetrics and Gynecology
Williams,Salvatore,574,NRS,Neurosurgery

Number of unmatched residents: 1
Number of positions available: 1
```

The residents are listed here in alphabetical order, but this is not a requirement. Note that, in this example, resident Tom Tan has not been matched. As shown above, we also ask you to print to the console two additional lines giving the number of residents that have not been matched and the total number of positions that are still available in the programs that have not reached their quota.

Programming

You will be asked to write programs implementing the solution to the stable marriage problem under different paradigms. Specific instructions for each language will be given to you.

Each program will be marked as follows:

- Program produces the correct result [3 points]
- Adherence to programming paradigm [2 points]
- Quality of programming (structures, organisation, etc) [1.5 point]
- Quality of documentation (comments and documents) [1.5 point]

You must submit all your source files together with the outputs of your program (in a text file). All your files must include a header showing student name and number. These files must be submitted in a zip file. This zip file must be named after your student number, for example if you student number is 123456 then the zip file for the object-oriented programming component will be `123456oo.zip`.